

MRK18 · BUILD PLAYBOOK

Build Your Own Posting Engine

Replicate what Ayrshare does for ~\$0/month in middleware — by filling in the scaffolding MRK18 already has. Step-by-step, build-ready, scoped to the 10-user MVP (LinkedIn first).

Prepared 23 June 2026 · AI-CMO · LinkedIn personal posting + scheduler first · approve-then-auto

MRK18 — Build Your Own Posting Engine (DIY, no Ayrshare)

A detailed, build-ready playbook to replicate what Ayrshare does — **for ~\$0/month in middleware** — by filling in the scaffolding MRK18 already has. Scoped to the 10-user, investor-ready MVP: **LinkedIn personal posting + scheduling first**, the rest deferred. Prepared 23 June 2026.

0. The core idea (why this is affordable)

Ayrshare is a paid **middleman** that does 4 jobs so you don't have to:

- 1. Holds a developer app with each platform.
- 2. Runs the "Connect your account" OAuth flow per user + stores tokens.
- 3. Calls each platform's **posting** API on the user's behalf.
- 4. Calls each platform's **analytics** API + refreshes tokens.

You can do all 4 yourself. The reason it's cheap for 10 users: **the only platform you need first — LinkedIn personal-feed posting — is free and self-serve (no approval)**. You pay nothing for middleware; the only platform fee is X's pay-per-use (pennies/post), which you'd pay either way.

Cost: ~\$0/mo middleware + X's per-post pennies + dev time (vs Ayrshare ~\$599/mo).

1. What you already have (≈80%)

Capability	Where it lives	Status
Encrypted token vault (access + refresh, per user × platform)	<code>security/vault.py</code>	✅ done
OAuth code-exchange (connect flow)	<code>security/oauth.py</code>	✅ done (needs a refresh fn)
Publisher adapters, one per platform	<code>publishers/adapters.py</code>	⚠️ stubbed — fill in
Idempotent publish (<code>request_id</code> + unique constraint)	<code>publishers/base.py</code>	✅ done
Approval gate (signed, re-verified at publish)	Gate-2 + <code>verify_approval()</code>	✅ done
Background worker (arq + Redis cron)	<code>worker/main.py</code>	✅ done (add 2 crons)

So the build = fill in the real per-platform API calls + create the (free) developer apps + add a scheduler table and two cron jobs.

2. Who does what

Some steps only **you** (the founder) can do — they need your identity/login. The rest is **code** (we build together).

Step	Founder (you)	Code (we build)
Create MRK18 LinkedIn Company Page	✓ (login required)	—
Register LinkedIn / X / Reddit developer apps	✓ (your account)	—
Add X developer billing (pay-per-use)	✓ (payment)	—
Put client IDs/secrets into config/secrets	✓ (paste values)	scaffolding
OAuth connect flow, token refresh	—	✓
LinkedIn posting adapter (API calls)	—	✓
Scheduler table + cron jobs	—	✓

3. STEP 1 — Create the developer apps (one-time, mostly free)

3a. LinkedIn (the priority — free, self-serve, no review)

1. Create a **LinkedIn Company Page** for MRK18 (free) — required to own a developer app.
2. Go to the **LinkedIn Developer Portal** → **Create app**; associate it with that Page; verify the Page (a Page super-admin clicks "Verify").
 1. Under the app's **Products** tab, add (both are **self-serve, instant, no review**):
 - **"Sign In with LinkedIn using OpenID Connect"** → grants scopes `openid`, `profile`, `email`.
 - **"Share on LinkedIn"** → grants scope `w_member_social` (the posting permission).
 1. Copy the app's **Client ID** and **Client Secret**. Set the **Authorized redirect URL** to your backend callback (e.g. `https://api.mrk18.app/oauth/callback`).
 1. App name/logo must **not** contain "LinkedIn" or "Microsoft."

Result: you can post to any member's personal feed once they connect — **no partnership, no fee**. (Company-Page posting and Ad-data need separate slow approvals — **defer both**.)

3b. X / Twitter (optional for MVP — costs money)

1. Create an **X Developer account**, create a **Project + App**, enable **OAuth 2.0** (user-context).
2. Add **billing** — X is now **pay-per-use** (~\$0.015/post, **\$0.20 if the post has a link**, reads \$0.005). You pay X directly; no middleman.
 1. Scopes: `tweet.read`, `tweet.write`, `users.read`, `offline.access` (for refresh tokens).
 2. Copy Client ID/Secret; set the redirect URL.

3c. Reddit (recommend: skip auto-post for MVP)

1. reddit.com/prefs/apps → create a **"web app"** (free) → get client_id/secret.
2. Submit Reddit's **mandatory pre-approval** form (required since Nov 2025).
3. Scopes: `identity`, `submit`, `read`, `history`. User-Agent must be `web:com.mrk18.app:v1.0.0 (by /u/<you>)`.

Recommendation: don't auto-post to Reddit in the MVP (spam rules → banned customer accounts). Use draft-and-assist (AI writes, human posts). Build this last, if at all.

4. STEP 2 — Wire config + the connect flow + token refresh

4a. Config (`config.py`)

Add per-platform settings (from env / secrets store — never in git): `LINKEDIN_CLIENT_ID` , `LINKEDIN_CLIENT_SECRET` , `LINKEDIN_REDIRECT_URI` (and the same for X, Reddit).

4b. Connect flow (mostly exists in `oauth.py`)

1. **Start:** `GET /founders/{id}/connections/linkedin/start` → build the LinkedIn authorize URL with `client_id` , `redirect_uri` , `scope=openid profile email w_member_social` , a random `state` (CSRF) → redirect the founder there.

1. **Callback:** `GET /oauth/callback?code= ... &state= ...` → exchange the `code` at LinkedIn's token endpoint for `access_token` + `refresh_token` → store encrypted via the existing vault (`store_tokens`), keyed by founder × platform.

4c. Add the ONE missing piece — `refresh_access_token()` in `oauth.py`

The vault stores refresh tokens but there's no refresh call yet. Add:

```
async def refresh_access_token(provider, refresh_token) → new tokens
```

- LinkedIn access tokens last **60 days**; refresh tokens last **365 days**.
- Call it **proactively** right before publishing if the token expires within ~5 min; **reactively**

if a publish returns 401. Re-store the refreshed tokens.

- Plan a **once-a-year "reconnect LinkedIn" nudge** (refresh token expires at 365 days).

5. STEP 3 — Fill in the LinkedIn posting adapter (`publishers/adapters.py`)

This is the real work (~a few days). Implement, using the connected member's access token:

1. **Get the author URN** (once, cache on the connected account):

`GET https://api.linkedin.com/v2/userinfo` → `sub` → author = `urn:li:person:{sub}` .

1. **Text post:** `POST https://api.linkedin.com/v2/ugcPosts`

- Header: `Authorization: Bearer <token>` , `X-Restli-Protocol-Version: 2.0.0` .
- Body: author URN, `lifecycleState: PUBLISHED` , `shareMediaCategory: NONE` , the commentary text.
- Success = `201` ; the post id is in the `x-restli-id` response header.

1. **Image post (3-step upload):**

2. **Register:** `POST /v2/assets?action=registerUpload` with

`recipes: ["urn:li:digitalmediaRecipe:feedshare-image"]` , owner = author URN → returns an `uploadUrl` + an `asset` URN.

1. **Upload:** POST the raw image bytes to that `uploadUrl` with the Bearer token.

2. **Post:** `POST /v2/ugcPosts` with `shareMediaCategory: "IMAGE"` and

`media: [{ status: "READY", media: "<asset URN>" }]` .

1. **Idempotency (reuse the existing pattern):** before posting, check/insert a `publish_results`

row keyed by the unique `request_id = "<scheduled_post_id>:linkedin"` . If it already exists with a terminal status, skip — a retry/crash can never double-post.

1. **Rate limits:** 150 posts/member/day, 100k/app/day — far above 10 users.

2. **ToS:** official API + human-approved content only. **Never scrape or browser-automate** — that gets the app + Page banned.

Do the **X adapter** the same way later (OAuth 2.0 user token → `POST /2/tweets`), remembering the per-post fee. Reddit adapter last (or skip).

6. STEP 4 — The scheduler (plan-ahead + auto-publish)

6a. New tables (`db/models.py` , keep RLS by `founder_id`)

```
content_calendar(  
    calendar_id PK, founder_id FK, run_id FK?, name, horizon_days,  
    timezone (IANA, e.g. Asia/Kolkata), approval_state (draft|approved|paused),  
    approved_at, created_at, updated_at)  
  
scheduled_posts(  
    scheduled_post_id PK, founder_id FK, calendar_id FK, item_id FK (the draft body/media),  
    platform, scheduled_at (timestampz, UTC), status (draft|scheduled|publishing|published|failed|retryable),  
    approval_state, approval_event_id FK (the signed Gate-2 approval),  
    request_id (unique idempotency key), attempts, max_attempts, next_attempt_at, last_error,  
    publish_result_id FK, locked_at, created_at, updated_at)  
index on (status, scheduled_at) where status in ('scheduled','retryable')
```

Also add `connected_accounts.last_metrics_fetch_at` (weekly-fetch cursor). `signals` + `analytics_diagnoses` need no change.

6b. The "publish due posts" cron (`worker/main.py`)

Add one arg cron, **every minute**:

1. `SELECT ... FROM scheduled_posts WHERE status='scheduled' AND scheduled_at <= now() ORDER BY

scheduled_at LIMIT N FOR UPDATE SKIP LOCKED` (SKIP LOCKED = safe with >1 worker later).

1. Set `status='publishing'` → **refresh token if near expiry** → call the platform adapter →

`verify_approval()` re-check → write `publish_results` (unique `request_id`).

1. Success → `status='published'` . Failure → `status='retryable'` , `attempts++` ,

`next_attempt_at` = exponential backoff w/ jitter (1m→5m→30m→2h, max ~5), then `failed` .

1. Store times as **UTC**; convert to the founder's IANA timezone only at the UI edge.

7. STEP 5 — Approve-then-auto (reuse Gate-2, keep trust)

Do **not** post fully unattended (ToS + brand safety). The model:

1. AI (`ad_copy` adapter) drafts the 10/20/30-day calendar → `content_items` + `scheduled_posts`

(`approval_state=pending`).

1. **Approve the whole calendar once** via the existing Gate-2 → mints a **signed ApprovalEvent**

per post → `status=scheduled` .

1. **Edit/pause window**: until ~30 min before send, the founder can edit/reschedule/pause (just an

`UPDATE`). Editing a post invalidates its approval signature → quick one-tap re-approve (this is enforced automatically by `verify_approval()` at publish time).

1. The publish cron only ships signed-approved posts — even a bug can't post unapproved content.

8. STEP 6 — Analytics (organic now, ad-data later)

- **Now (organic):** a weekly arq cron per connected account pulls recent post performance →

`signals` (post-level) + an account rollout → `analytics_diagnoses`. Staggered per tenant, incremental via `last_metrics_fetch_at`, honor `Retry-After`.

- **Ad data (defer):** LinkedIn Ad reporting needs the Marketing API approval (weeks–months). For the demo, accept a **one-time manual CSV import** of past ad data instead. Make live ad-ingest a roadmap slide.

- Note: LinkedIn **personal** post-analytics has **no API** — only org Pages (with approval) do. So "post performance" via API is limited; lean on organic engagement you can read + the manual import.

9. Build order (what we'll do when we start)

1. **Foundation (platform-agnostic, safe to build now):** `content_calendar` + `scheduled_posts` tables + the `refresh_access_token()` function. *(Half a day.)*

1. **LinkedIn app:** you create the Company Page + dev app + add the two self-serve products; paste client ID/secret into config. *(You — ~1 hour.)*

1. **LinkedIn adapter:** real `userinfo` + `ugcPosts` text post, then the 3-step image upload.

(A few days.)

1. **Connect flow end-to-end:** start → callback → token stored → one manual test post. *(½ day.)*

2. **Publish-due-posts cron** + idempotency + backoff. *(½ day.)*

3. **Approve-the-calendar (Gate-2 reuse)** + edit window. *(½–1 day.)*

4. **Weekly organic fetch cron.** *(½ day.)*

5. **X adapter** (optional, when you add X billing). Reddit: draft-assist only / skip.

MVP demo after steps 1–6: **"AI drafts a month of LinkedIn posts → I approve once → they auto-publish on schedule."** Live, honest, **\$0/mo middleware**.

10. What you give up vs Ayrshare (be honest)

- **You maintain the adapters** — platforms occasionally change APIs; you patch them.
- **The slow approvals are yours** — LinkedIn Ad-data + Company-Page posting (deferred anyway).
- **More upfront dev time** — but no \$599/mo bill, no vendor lock-in, full control, and it's

attainable today for the MVP.

- Revisit a paid unified API only at **a few hundred users**, when your time is worth more than the fee and you have the revenue to cover it.

Quick reference — LinkedIn (the MVP platform)

- **Scopes:** `openid`, `profile`, `email`, `w_member_social` (all self-serve).
- **Author:** `GET /v2/userinfo` → `urn:li:person:{sub}`.
- **Post:** `POST /v2/ugcPosts` (header `X-Restli-Protocol-Version: 2.0.0`); id in `x-restli-id`.
- **Image:** registerUpload → upload bytes → reference asset URN.
- **Tokens:** access 60 days, refresh 365 days; reconnect ~yearly.
- **Limits:** 150 posts/member/day. **Rule:** official API + human-approved content; never scrape.